Logan Rayburn

Assuming you already have a Server, Database and Tables:

Open Visual Studio. Click “Create new project” and select ASP.NET Core Web API. After that, feel free to leave everything default. Now, start by downloading Microsoft.EntityFrameworkCore.Tools, .Design and .Sqlserver. Do this by clicking “Project” then go down near the bottom of the list and select Manage NuGet Packages. It will be a light blue icon.

After you have those, go to appsettings.json and below Allowed Hosts, put

```
 "ConnectionStrings": {
          "DefaultConnection":
         }
```

Then, go to “View” in the top bar, select “SQL Server Object Explorer. If you do not see your server, click SQL Server, Add SQL Server and type in the server name. Make sure “Authentication” is WIndows Authentication, and check the box at the bottom for Trust Server Certificate. Then click Add.

Now, go back to SQL Server Object Explorer and click the arrow next to your SQL Server. Right click your database, click properties and look for “Connection string”. Copy that, and place that inside quite at the end of the code detailed earlier, but still inside the brackets.

Now, go to “Tools”. Hover over Nuget Package Manager and click Package Manager Console. Inside that, type in scaffold-DbContext “CONNECTION STRING” Microsoft.EntityFrameworkCore.SqlServer . This will make your DbContext. Then make a Data folder, and place the Context item in that folder. THe connection string has to be in quotes. If it is not working, close and reopen SSMS, go and recopy the connection string and try that again.

Go to launchSettings, and at launchBrowser set it to true and add “launchUrl” below it and set it to whatever you want launched. You get the name for this from the COntrollers which we wil make in a moment.

Now, go to Program and above builder.services put this code there:

```
builder.Services.AddDbContext<YOUR CONTEXT>(options =>
   options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

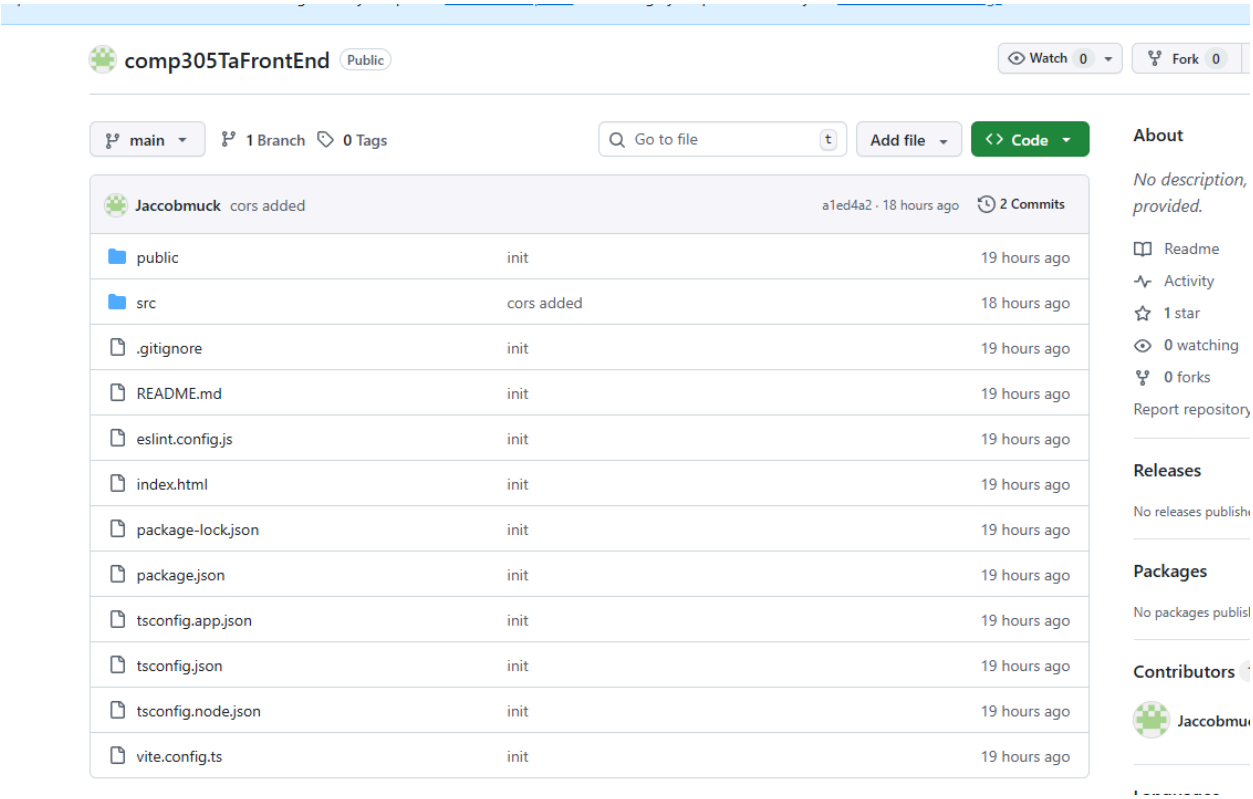
At the top of Program, put “using (your program).Data;

Now, for organization move the new items that should have appeared into a new folder called Models. CLick Ctrl, and left click each item then drag them to Models, and allow for namespace refactoring.

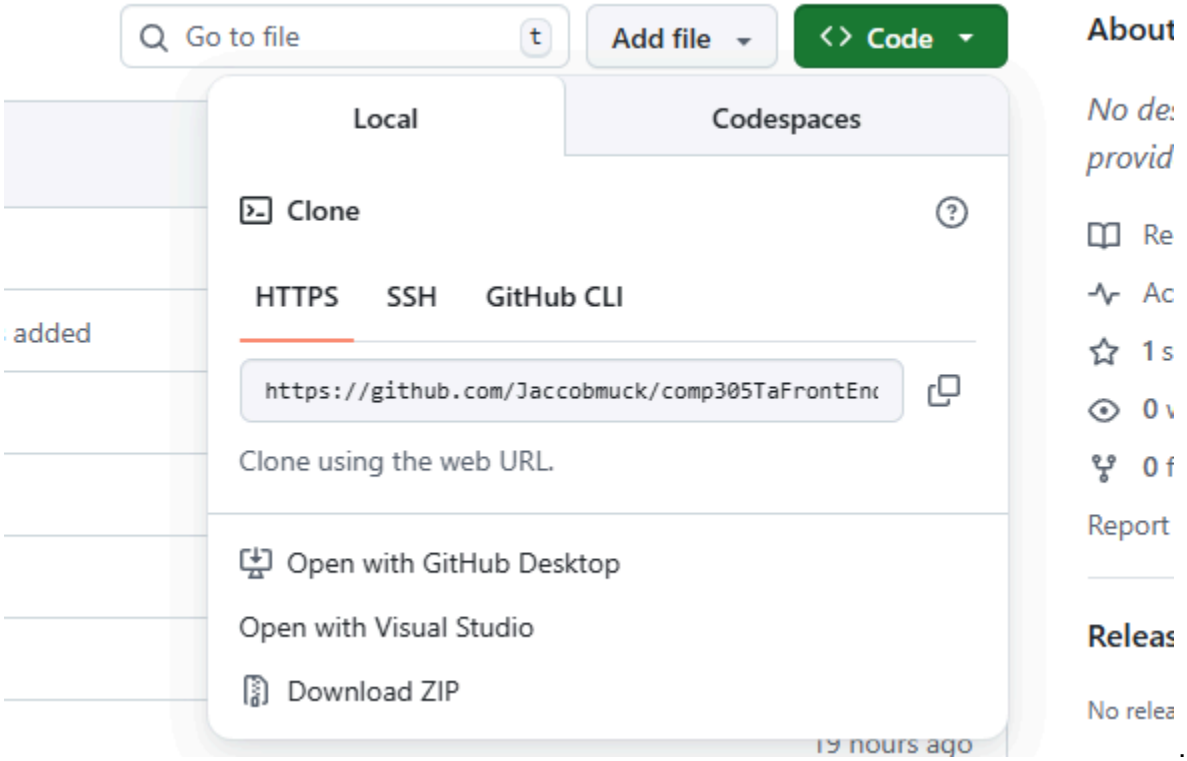
Next, go to Controllers. Right click that and hover over Add and click Controller. Go to API and click API Controller with actions, using Entity Framework. Click Add, find the item you want to make a Controller for, set the DbContext class to the Context folder that was made. Do that for every item you need.

The following is just an example, the actual GIthub link is after this:

For front end: Go to GitHub, “https://github.com/Jaccobmuck/comp305TaFrontEnd”. Click the big green Code button.



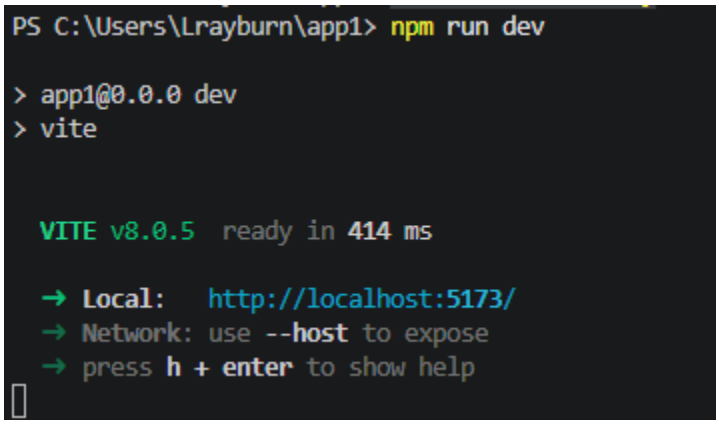
Then, copy the URL it provides.



Now, go to Visual Studio Code. Open the Terminal and type “git clone https://github.com/Jaccobmuck/comp305TaFrontEnd.git”.

Then, type “npm run dev”. If you receive this error:
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see about_Execution_Policies at https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm run dev
+ ~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
TO stop running something in the terminal, press control c while in the terminal.

Then type “Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned”. Now, try again with the npm line. If it worked, you should see something like this:



Current versions are stored in GIthub at:
https://github.com/Frostbyte1/Software_Engineering_Final_Correct

The previous link was just an example.

Frontend:

April_7 is the backend and goes to Visual Studio. To open, open Visual Studio, create a new project, then set up programs and steadily transfer over programs, manually. The frontend is in Visual Studio Code, and is app1. The main programs are inside that file, specifically a folder named “src”. The database is Orderscript.sql, and is in SQL Server Management Studio, and can literally just have the code copied into a new query.

The frontend of any program is the user-facing part of the program. It has no direct connection to the database, rather it connects to the backend, and the backend queries the database for the relevant information, receives that data, and sends it to the frontend. The frontend communicates with the backend via endpoints.

Some reasons for the existence of a frontend are A: there is plenty of information a user shouldn't be able to access, like private information on other users. B: Frontends don’t display information through something like JSON like backends do, rather they use UIs both to receive inputs from the user and to display the information they want in a cleaner and more aesthetic format.

As for how this works.

On line 15, we have this: `const API_URL = 'http://localhost:5045/api/Orders'` which is basically telling the frontend where to send its requests, i.e. the backend. You get this from the URL that generates from the backend running.

Next are the CRUD functions themselves. CRUD just means Create, Read, Update and Delete. Nothing complicated, basically just being able to make new data, read existing data, update the data (like editing existing data) and delete some data.

Frontend functions don't actually retrieve/ update or do anything to data directly. Rather, the frontend takes a request from the user, sends it to the backend via the URL found on line 15 in this case, the backend takes that request, sends the relevant request to the database, receives the data and the backend sends the data to the frontend using an API endpoint which then displays it to the user via a UI.

The frontend can be thought of as the actual app someone downloads, and the backend is the program on the company servers, hence has direct connection to the databases.

While there is more code related to each, these are the basics for each of the CRUD functions.

READ:

```
useEffect(() => {
  getOrders()
}, [])
```

CREATE:

```
await fetch(API_URL, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(orderToSend)
})
```

UPDATE:

```
setEditingId(order.orderId)
```

DELETE:

```
await fetch(`${API_URL}/${id}`, {
```

```
  method: 'DELETE'
})
```

The Update and Delete functions are triggered by a button, which looks something like this:
onClick={() => handleDelete(order.orderId)}

Again, most of the code for these functions is handled by the backend. In part, I believe this is to minimize the file size the user is downloading, but also to help streamline processes. This is also because having the program doing the actual work directly connected to the relevant database is faster than sending the full function wirelessly from a user's phone.

AN example of some code:

```
useEffect(() => {
   getOrders()
 }, [])

 async function getOrders() {
   try {
     setLoading(true)
     setError('')

     const response = await fetch(API_URL)

     if (!response.ok) {
       throw new Error('Failed to fetch orders')
     }

     const data = await response.json()
     setOrders(data)
   } catch (err) {
     setError(err instanceof Error ? err.message : 'Unknown error')
   } finally {
     setLoading(false)
   }
 }
```

THis line: `const data = await response.json()`

```
      setOrders(data)
```

Translates the JSON given by the backend and converts it into JavaScript data, then stores it in “orders”.

The result is displayed with: orders.map((order) => ...)

Code like this: `<button type="submit">`

```
          {editingId === null ? 'Add Order' : 'Update Order'}
        </button>
```

Is actually fairly simple. The question mark is basically an if statement. If the previous statement is true, then the bit right after it will be done. The “:” is an “else” statement. If the previous statement is false, then do what is after the “:”. In practice, by default editingId is null and as such the button is set to Add Order. If an Edit button is clicked, then editingId will be set to the Id of the order in question, but for our uses the specific Id is unimportant, just that the Id is not null now. As such, the Add Order button is changed to Update Order.

```
const [orders, setOrders] = useState<Order[]>([])
```

useState allows React to remember values between page updates, things like inputs, errors or loading messages. “const” makes a variable that is supposed to stay the same. “orders” is the data already present, and setOrders is used to change the data in question. In practice, this allows React to remate the UI based on changing data, in this case regarding the Orders.

```
useEffect(() => {
    getOrders()
  }, [])
```

useEffect basically serves as the start for where the program should run. useEffect says to run “side-effect” code, things like fetching data, timers and others. getOrders is the “main” code it is supposed to run, and [] tells it to stop running after running once.

```
const response = await fetch(API_URL)
```

This is the sort of code that actually sends requests to the backend, hence the API_URL. Essentially, it “fetches” data from the backend. “await” tells the code to hold off on running until it receives a response from the backend.

```
function handleChange(e)
```

This handles keeping track of the input the user gives, or every time the user types.

```
setEditingId(order.orderId)
```

This allows for the app to move into editing mode, in this instance meaning the same form can now either add a new order or edit an existing one. It is more than just this line, but this allows for the switch.

```
 {loading && <p>Loading orders...</p>}
```

This is very simple. && means a conditional loading, basically if something is happening then display the text after. In this case, if the orders are being loaded, the display “Loading orders…”

```
 {orders.map((order) => (
```

This means to loop through all orders, and make an HTML row for each one. Specifically the .map component, like you’re “mapping” the info of each order onto the UI.

```
handleSubmit
```

handleSubmit is responsible for sending form data to the backend. If editingId is null, it creates a new order using POST. Otherwise it updates an existing order using PUT. Afterward it refreshes the order list by calling getOrders().

A basic visual for how this project functions is as follows:

User
↓
React Frontend
↓ HTTP Request
ASP.NET Backend
↓ SQL Query
SQL Server Database
↓
ASP.NET Backend
↓ JSON Response
React Frontend
↓
Updated UI